```
>>> 1/3
0.3333333333333333
>>> 1/3*3
1.0
>>> type(1/3)
<class 'float'>
>>> type(0.3333333333333333)
<class 'float'>
>>> x^2 + 2*x + 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'x' is not defined
```

Figure 1: A few Python one-liners

```
sage: 1/3
1/3
sage: 1/3*3
1
sage: type(1/3)
<class 'sage.rings.rational.Rational'>
sage: type(0.3333333333333333)
<class 'sage.rings.real_mpfr.RealLiteral'>
sage: x^2 + 2*x + 1
x^2 + 2*x + 1
```

Figure 2: A few SageMath one-liners

Now, let us compare the outputs produced by each one-liner shown in Figures 1 and 2. The first one-liner performs a simple division operation: *1/3*. In Python 3, this results in a floating-point approximation: *0.3333333333333333*. In contrast, SageMath interprets both operands as elements of the rational number field Q, and thus retains the result as the exact fraction *1/3* unless explicitly converted. This seemingly simple example illustrates a core philosophical distinction between the two systems: Python emphasises general-purpose programming with a focus on practical computation, while SageMath is designed for mathematical fidelity, preserving exact structures wherever possible.

Now, let us examine the second one-liner: the expression *1/3 * 3*. In Python 3, the / operator performs floating-point division, so *1/3* evaluates to *0.3333333333333333*. Multiplying this by *3* gives *0.9999999999999999*, which is approximately equal to *1.0*. This discrepancy arises due to the inherent limitations of floating-point arithmetic, where certain decimal values cannot be represented exactly in binary form. In contrast, SageMath treats numbers like *1/3* as exact rational numbers. So the expression *1/3 * 3* evaluates to exactly *1* without any rounding or approximation. This exactness is particularly useful in mathematical computations where precision is important, such as in symbolic algebra or number theory.

Now, consider the third and fourth one-liners involving *type(1/3)* and *type(0.3333333333333333)*. In Python 3, both expressions are interpreted as floating-point numbers, and hence their type is *float*. This reflects Python 3's default behaviour of using floating-point arithmetic for division. In SageMath, however, *1/3* is stored as an exact rational number and its type is *Rational*. On the other hand, *0.3333333333333333* is treated as an approximate real number, and its type is *RealLiteral*, indicating that it is a floating-point approximation.

Finally, the fifth one-liner in Python raises an error because Python doesn't know what *x* is unless it is defined, and the character ^ represents *bitwise XOR* in Python, not exponentiation. In SageMath, the same one-liner works fine, as it interprets *x* as a symbolic variable and ^ as exponentiation.

These one-liners illustrate how SageMath enriches Python with deep mathematical knowledge, making it far more than just a wrapper or extension—it is a full-fledged system for mathematical computation.

Let us now look at some examples where SageMath uses built-in mathematical functions that are not available in standard Python without importing external packages. The one-liners in Figure 3 demonstrate SageMath performing relatively complex mathematical operations with ease.

```
sage: Mod(98765432198765433198765432O, 987654321987654321987654321)
9999999999
sage: is_prime(10000000000000000000000000000000000037)
True
sage: factor(1000000016000000063)
1000000007 * 1000000009
sage: lcm(1000000009, 1000000007)
1000000016000000063
sage: gcd(1000000009, 1000000007)
1
sage: QQbar(sqrt(2))
1.414213562373095?
```

Figure 3: Some SageMath functions

Now, let us understand the working of these SageMath functions. The SageMath commands in Figure 3 effectively demonstrate the system's robust capabilities in symbolic and numeric computation, particularly in number theory and algebra. The initial command, *Mod(a, b)*, swiftly computes the remainder of a very large number modulo, showcasing SageMath's efficient handling of big integers. Following this, the function *is_prime( )* reliably confirms that *10000000000000000000000000000000000037* is indeed a prime number. The function *factor( )* then expertly decomposes *1000000016000000063* into its prime components, *1000000007 * 1000000009*, both being large primes. Subsequent operations, *lcm( )* and *gcd( )*, further illustrate SageMath's power: *lcm( )* computes the least common multiple of these two primes, returning the original composite number, while *gcd( )* confirms their coprimality (a property of two numbers that have no common factors other than 1) by yielding 1. Beyond integer arithmetic, the